

Trabalho 2

Avaliação de modelos

O Trabalho 1 envolveu a definição do problema a ser trabalhado, a coleta de dados, sua análise exploratória, definição da abordagem, métricas e métodos de avaliação e spot-checking, para avaliação preliminar de modelos. Isso permitiu o ganho de familiaridade com o problema e determinação de quais modelos podem ser explorados mais a fundo.

A proposta do Trabalho 2 é exatamente realizar a análise mais aprofundada dos modelos mais promissores encontrados no Trabalho 1 e na comparação entre eles.

Tarefas

1. Treinamento e Avaliação dos Modelos

Cada modelo escolhido deve ser treinado para obter o melhor desempenho possível. Não havendo um referencial de desempenho para o problema, apenas obtenham o melhor modelo dentre os que vocês testaram. Isso envolve o melhor ajuste de hiperparâmetros que for possível encontrar. Automatizar essa etapa pode economizar bastante esforço. O scikit-learn oferece o GridSearch que é uma busca exaustiva trivial. Assim, bibliotecas que implementam algoritmos mais “inteligentes”, como Hyperopt¹ e Optuna², podem ser um diferencial na obtenção de um bom desempenho.

Garanta que a metodologia de pré-processamento, treino e validação esteja correta. Um aspecto particularmente crítico é a prevenção de data leakage, que pode comprometer os resultados e a validade do modelo. Por exemplo, o cálculo das estatísticas de pré-processamento (e.g. fit do scikit-learn para normalização, balanceamento de dados e imputação) só pode ser feito nos dados de treino. Esse cálculo é, então, aplicado (e.g. transform do scikit-learn) nos dados de treino, validação e teste. Além disso, a otimização de hiperparâmetros deve ser avaliada no conjunto de validação e não no de teste.

A divisão dos dados em treino, validação e teste pode ser feita via holdout de 3-vias ou holdout+validação cruzada (CV), onde o holdout separa o de teste e o restante é dividido conforme a validação cruzada, ou validação cruzada (CV) aninhada. Avalie qual a melhor estratégia, balanceando a confiabilidade do método (e.g. validação cruzada aninhada garante mais variabilidade nos dados) vs o tempo de execução (e.g. 5-folds externos e 3-folds internos de CV aninhada fazem com que cada possível modelo seja treinado e avaliado 15 vezes). Independente do método escolhido, a divisão dos dados deve ser estratificada (e.g. mesmo percentual de cada classe no treino, validação e teste).

¹ <http://hyperopt.github.io/hyperopt/>

² <https://optuna.org/>

Utilize as métricas definidas na primeira parte do trabalho para comparar os modelos. Garanta que a métrica seja a correta para o problema, considerando eventuais desbalanceamentos nos dados.

2. Comparação de Modelos

- **Análise de Desempenho:** após otimização de hiperparâmetros de todos os modelos candidatos, compare seu desempenho nos dados de teste para identificar qual modelo oferece o melhor desempenho global e quais são os pontos fortes e fracos de cada modelo.
- **Visualizações:** Crie gráficos e visualizações que ajudem a ilustrar as comparações entre os modelos. Conforme apropriado, obtenha curvas ROC ou PR, gráficos de barras comparando as métricas de cada modelo, e/ou matrizes de confusão.
- **Análise:** identifique o melhor modelo, ou se não há um modelo claramente melhor, quais são os “vencedores” e as situações e características de vantagem e desvantagem entre eles. Verifique os casos (amostras) onde o melhor modelo (ou os melhores, em caso de “empate”) vai(vão) bem ou mal. A análise não deve ser pra cada amostra individual, mas de maneira agregada. Por exemplo, se o problema for de classificação, analise qual é a classe mais difícil para o modelo, e se há amostras “confusas” no sentido que seus atributos parecem realmente pertencer a outra classe. Se o problema é de regressão, veja se há algum padrão onde os erros são maiores.

Entrega

Os grupos deverão enviar um arquivo .zip pelo moodle, contendo:

1. **O relatório em PDF.** O relatório do Trabalho 2 deve ser uma continuação do relatório do Trabalho 1. Assim, comece o relatório do Trabalho 2 com tudo que já foi feito no Trabalho 1 (as descrições e conclusões relativas à (1) coleta de dados e identificação do problema; (2) análise exploratória dos dados; (3) definição da abordagem, métricas e métodos de avaliação e (4) spot-checking). Se algo mudou em algumas dessas etapas, atualize as informações. Em adição a isso, crie as seguintes seções:
 - Treinamento e avaliação de modelos: onde você descreverá a metodologia de divisão de dados, métricas de avaliação e escolha de hiperparâmetros para cada algoritmo avaliado;
 - Comparação dos modelos: onde você apresentará a comparação de desempenho entre modelos, com os gráficos e visualizações e sua análise descritiva do que foi descoberto.
 - Conclusão: suas considerações finais sobre o trabalho, sobre o(s) melhor(es) modelo(s) obtido(s), desafios encontrados, e futuras possibilidades de continuação do trabalho.
2. O código fonte usado para concluir as etapas do Trabalho 2, e também o código usado nas etapas do Trabalho 1 (com atualizações, se foram feitas). O código deve estar em um subdiretório ‘src’ dentro do arquivo .zip. Python é a linguagem de escolha dos Deuses da IA, e, em especial, Jupyter Notebooks (e.g. Colab) são vistos como belas oferendas. Porém, o grupo pode implementar o código em qualquer linguagem de programação. Mas os Deuses da IA somente perdoarão isso se o código for enviado com instruções simples e precisas, em um arquivo Readme.md, de como compilar (se necessário), e rodar o código. Caso o código

tenha sido feito no Colab, baixe o .ipynb e envie no .zip. Não é válido o envio da URL do Colab.

Importante: caso a implementação seja no Colab, a simples exportação/impressão de um notebook como PDF não será aceito como relatório. O relatório deve ser um documento devidamente estruturado contendo as informações solicitadas acima.

A estrutura do arquivo `.zip` a ser entregue deve ser a seguinte:

```
relatorio.pdf
src          <-- subdiretorio contendo seu codigo
|-- Readme.md <-- com as instrucoes
\-- [arquivos e subdiretorios de sua implementacao]
```

Critérios de correção

- Completude do trabalho e atendimento aos requisitos definidos neste enunciado;
- Metodologia: avaliação de se a abordagem, métodos e as métricas escolhidas estão apropriadas;
- Qualidade do relatório final, em termos da estrutura, da redação e da discussão dos itens pedidos no enunciado.

Observações gerais

- O trabalho deve ser feito em grupos.
- Fiquem atentos à política de plágio!

Política de Plágio

O uso de assistentes de IA para programar é permitido, mas os grupos não podem copiar uns aos outros. Os grupos poderão apenas discutir questões de alto nível relativas a resolução do problema em questão. Poderão discutir, por exemplo, questões sobre os algoritmos utilizados, o ajuste de hiperparâmetros, etc.

Usamos rotineiramente um sistema anti-plágio que compara o código-fonte desenvolvido pelos grupos uns com os outros e com soluções enviadas em edições passadas da disciplina.

Qualquer nível de plágio entre grupos poderá resultar em nota zero no trabalho. Todos os envolvidos (não apenas os que copiaram) serão penalizados. Esta política de avaliação não é aberta a debate. Se você tiver quaisquer dúvidas se uma determinada prática pode ou não, ser considerada plágio, não assumam nada: pergunte ao professor(a) e monitor(es).

Note que, considerando-se os pesos das avaliações desta disciplina (especificados e descritos no Plano de Ensino), nota zero em qualquer um dos trabalhos tem grande impacto na nota, e pode impedir o acesso à prova de recuperação. Ou seja: caso seja detectado plágio, há o risco direto de reprovação.